

课程笔记

24 - 文件系统 API (1)

2026 南京大学操作系统原理

lecture-to-notes

2026-06-20



视频作者/频道：蒋炎岩，南京大学

发布日期：本地视频，发布日期未提供

视频时长：01:19:11

视频链接：未填写

目录

1 从块设备到文件系统	2
1.1 为什么不能只有文件编号	3
1.2 目录是带名字的指针表	4
1.3 本章小结	5
2 多棵目录树与挂载	5
2.1 Windows 与 Unix 的命名空间	5
2.2 mount: 把目录树贴到命名空间	6
2.3 ioctl: 设备控制的杂货铺	7
2.4 镜像文件如何被挂载: loop device	8
2.5 FHS: 根目录的共同语言	8
2.6 本章小结	9
3 目录树 API: 增删改查与链接	9
3.1 遍历与 glob	9
3.2 硬链接: 多个名字指向同一个文件	10
3.3 符号链接: 把路径写进一个特殊文件	10
3.4 Nix: 用链接拼出可回滚的软件世界	11
3.5 本章小结	12
4 文件元数据、权限与虚拟文件系统	12
4.1 ls -l: 属性的压缩视图	12
4.2 文件系统可以“说谎”: procfs	13
4.3 Extended Attributes: 给文件附加 key-value	14
4.4 ACL: 比 user/group/other 更细的授权	15
4.5 本章小结	16
5 总结与延伸	16
5.1 本章小结	16
5.2 拓展阅读	17

1 从块设备到文件系统

上一讲已经把存储设备抽象到一个很低的层次：块设备提供 `read block` 和 `write block`，必要时再提供等待落盘或查询状态的接口。这个接口非常小，但它把 SSD、HDD、SD 卡、光盘乃至内部带有 ARM 控制器的复杂设备都压缩成了一个大数据字节序列。问题在于，让应用直接共享这个字节序列不是好主意：多个进程如果都用 `open`、`mmap`、`lseek` 直接改同一个块设备，等价于多个进程无同步地共享一个巨大数据结构。

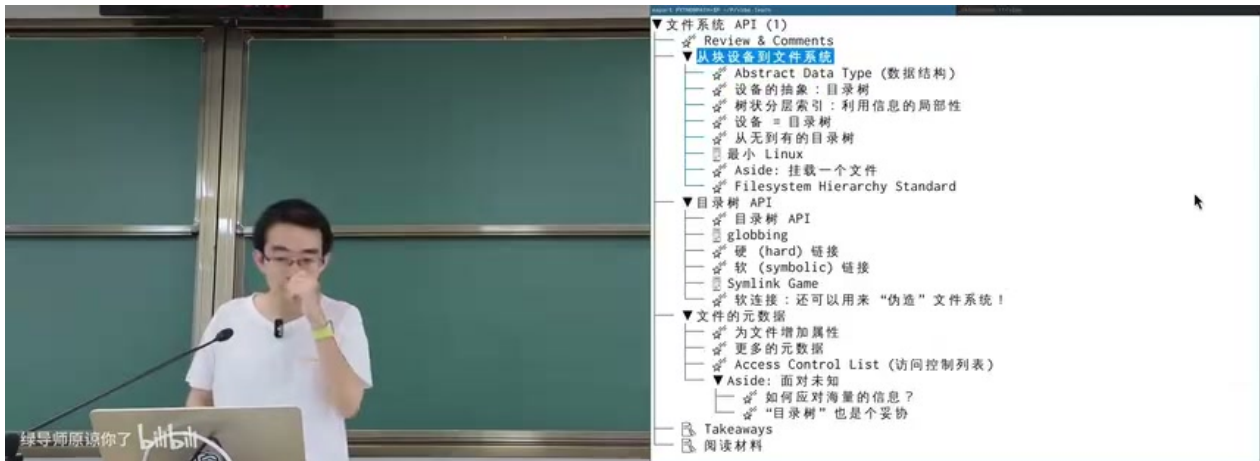


图 1: 本讲结构：把文件系统作为抽象数据类型，随后讨论目录树 API、链接、文件元数据和 ACL。¹

因此，文件系统的第一层意义不是“有文件夹图标”，而是为持久化字节序列提供可共享、可命名、可维护的数据结构。内存系统已经做过类似的事情：DRAM 是物理字节数组，MMU 把它变成每个进程独立的虚拟地址空间，`malloc/free` 再在地址空间上建立动态对象。块设备也类似：物理设备是字节数组，文件系统把它组织成若干可增长、可截断、可命名的虚拟磁盘，也就是文件。

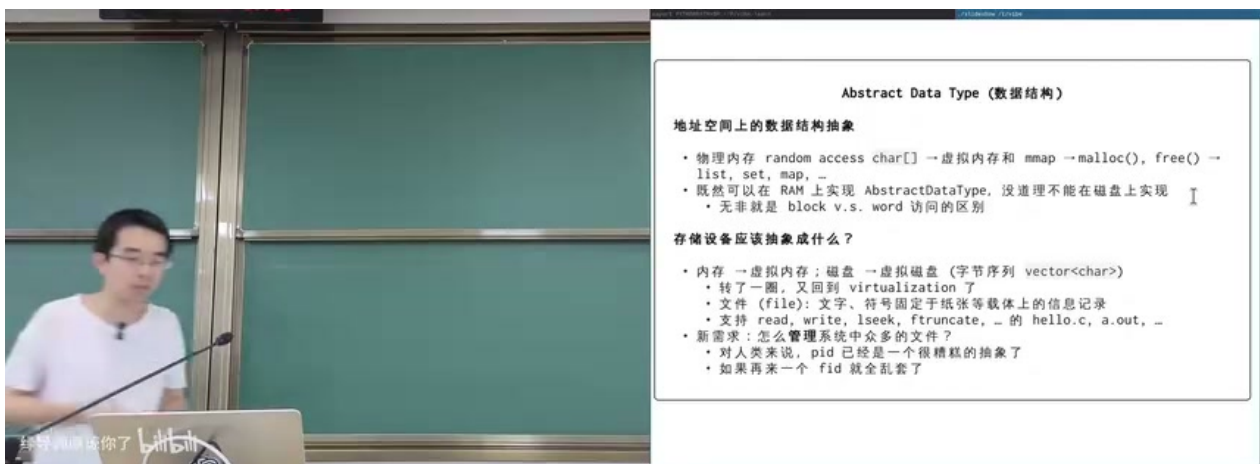


图 2: 文件系统的抽象：在物理字节数组之上构造虚拟磁盘、文件和目录树，而不是让进程直接共享块设备。²

¹ 视频画面时间区间：00:03:05–00:03:10。

文件就是可变长的虚拟磁盘

一个普通文件可以看成 `vector<char>`：它有长度，支持读、写、随机定位和截断。相比真实磁盘，文件可以变长；相比内存对象，文件具备持久性；相比裸块设备，文件由文件系统负责分配、命名、共享和回收。

文件层面的 API 已经在实验中反复出现：

```
1 ssize_t read(int fd, void *buf, size_t count);
2 ssize_t write(int fd, const void *buf, size_t count);
3 off_t lseek(int fd, off_t offset, int whence);
4 int ftruncate(int fd, off_t length);
```

Listing 1: 面向单个文件的核心系统调用

这些系统调用描述的是“一个文件内部”的行为。本讲的主角则是“整个系统里许多文件如何被管理”：目录树、路径名、挂载点、链接和元数据。

1.1 为什么不能只有文件编号

进程有 PID，文件系统对象也有类似的内部编号，例如 inode。理论上，如果 `open` 的第一个参数是一个整数文件 ID，系统仍然能工作。但对人来说，这样的接口非常糟糕：`/home/jyy/a.txt` 比 `176358` 更容易记忆、沟通和维护。文件系统给人的第一件礼物就是命名。

人类需要局部性

目录树的设计来自信息局部性：相关信息放在同一个子树中，用户可以沿着分类逐层缩小范围。图书馆按中图法分类，文件系统按目录分类，本质上都是把“查找”变成对层次索引结构的遍历。

²视频画面时间区间：00:03:20–00:03:25。

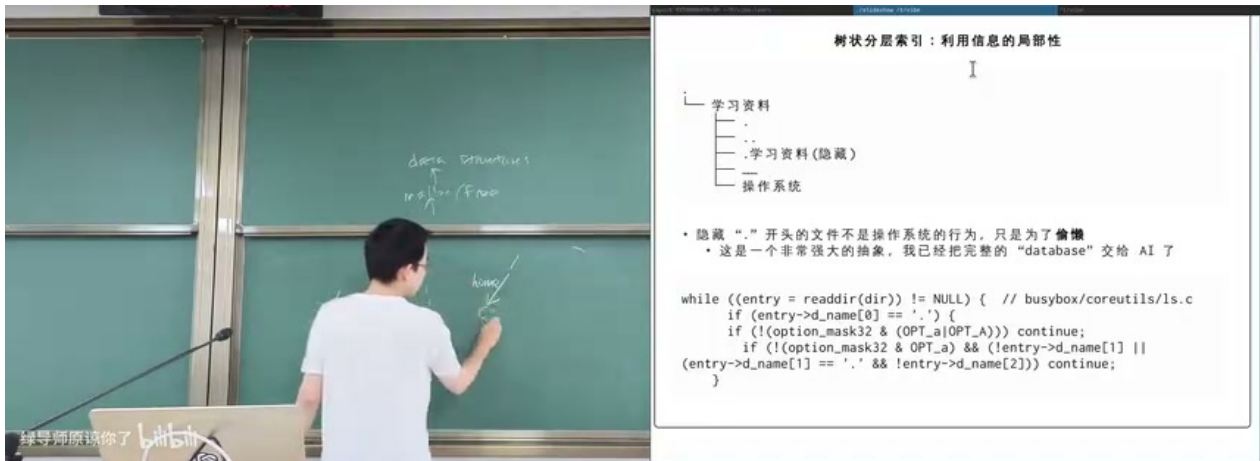


图 4: 隐藏文件的实现可以非常朴素: 目录项仍然存在, ls 这类工具只是在 readdir 后过滤以点开头的名字。⁴

不要把 UI 约定误认为内核语义

Unix 的隐藏文件主要是命名约定和工具行为。内核并不会因为文件名以 . 开头就改变其访问权限、生命周期或持久化语义。真正影响权限和执行的, 是后面要讲的 mode、owner、group、ACL 等元数据。

1.3 本章小结

块设备给操作系统一个大字节序列; 文件系统把这个字节序列组织成可命名、可共享、可回收的数据结构。普通文件是可变长的虚拟磁盘, 目录是名字到对象的映射表。目录树之所以适合人类使用, 是因为它把信息局部性编码进路径名。

2 多棵目录树与挂载

真实系统里不只有一棵目录树。硬盘分区、U 盘、光盘、网络盘、procfs、临时文件系统、镜像文件都可能暴露为目录树。操作系统必须提供一个机制, 把这些目录树接入统一命名空间。

2.1 Windows 与 Unix 的命名空间

Windows 用户熟悉 C:、D: 和网络共享路径, 但系统内部还有更底层的对象命名空间, 例如 \Device\HarddiskVolume1、\Driver\Ntfs、\??\C:。这些设计体现了历史兼容: A 盘和 B 盘来自软盘时代, C 盘以后才留给硬盘; \?? 又引入了 per-user 的命名空间。

⁴视频画面时间区间: 00:12:00–00:12:05。

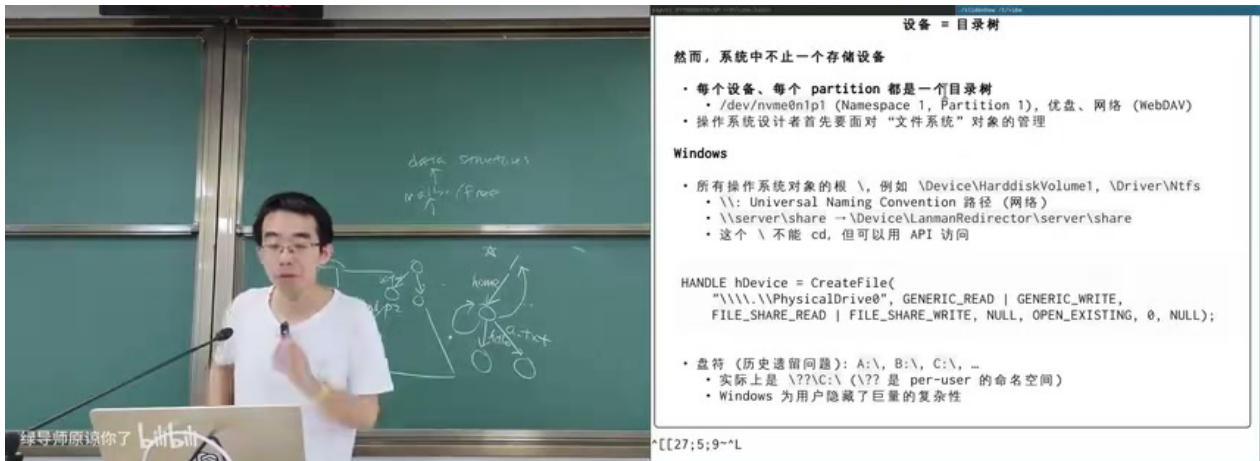


图 5: Windows 的用户可见盘符背后还有对象命名空间；盘符和网络路径只是底层对象路径的别名或重定向。⁵

Unix/Linux 的统一命名空间更直接：所有东西都被组织到一棵从 / 开始的树中。一个设备或虚拟文件系统要进入这棵树，就需要 mount。

2.2 mount: 把目录树贴到命名空间

mount 的语义可以一句话概括：把某个文件系统的根目录接到当前命名空间的某个目录上。从用户视角看，U 盘插入后桌面多了一个目录；从系统视角看，是某个块设备上的目录树被挂载到一个挂载点。

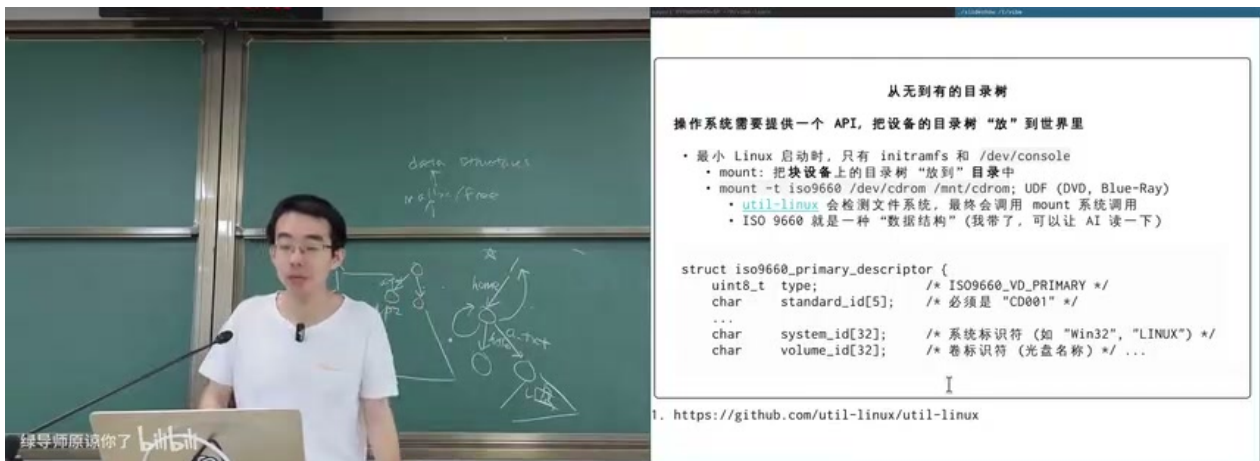


图 6: 从最小 Linux 到完整目录树：mount 把块设备或虚拟文件系统上的目录树放入统一命名空间；ISO 9660 是光盘文件系统示例。⁶

讲者现场插入 Win98 光盘，系统出现 /dev/sr0。光盘是只读设备，因此挂载时会出现 read-only 语义。ISO 9660 通过固定位置的数据结构识别文件系统，课堂中在十六进制视图中看到了 CD001 标记。

⁵视频画面时间区间：00:16:45–00:16:50。

⁶视频画面时间区间：00:21:25–00:21:30。

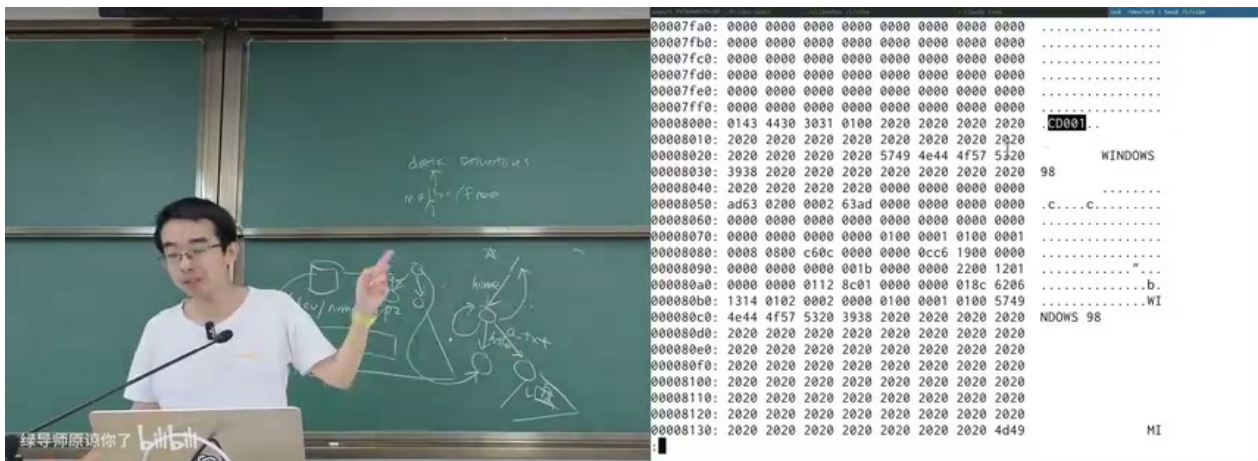


图 7: ISO 9660 光盘镜像中的 CD001 标记: 文件系统首先要在字节序列中定位可解析的数据结构根。⁷

文件系统驱动就是字节序列解析器

块设备只是字节数组。文件系统类型决定如何解释这些字节: 哪里是超级块, 哪里是目录项, 哪里是文件内容, 哪里是空闲空间管理结构。识别出格式之后, 内核调用对应文件系统驱动, 把字节数组投影成目录树。

2.3 ioctl: 设备控制的杂货铺

块设备最核心的操作是读写, 但现实设备还需要大量控制命令: 查询大小、检测光盘是否在托盘中、弹出光驱、设置 loop device 等。Unix 把这些非统一语义塞进 `ioctl`。这不优雅, 但很实用。



图 8: `eject` 光驱本质上调用 `ioctl(..., CDROMEJECT, ...)`; 设备控制不是魔法, 而是驱动暴露的命令接口。⁸

⁷ 视频画面时间区间: 00:25:40–00:25:45。

⁸ 视频画面时间区间: 00:30:30–00:30:35。

2.4 镜像文件如何被挂载: loop device

mount 系统调用期望源头是块设备,但很多实验和工具使用的是镜像文件,例如 fs.img。文件不是块设备,解决办法是创建 loopback device: 内核提供一个伪块设备 /dev/loop0,它的块读写被转发到普通文件的读写。

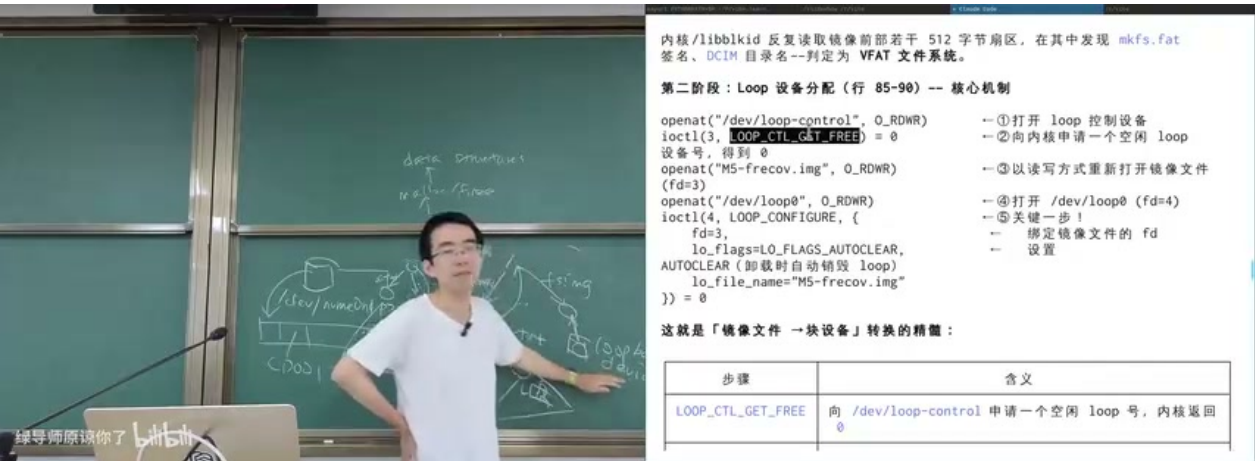


图 9: 挂载镜像文件的关键步骤: 通过 /dev/loop-control 申请空闲 loop 号, 将镜像文件绑定到 /dev/loop0, 再执行真正的 mount。⁹

这解释了为什么磁盘恢复实验可以把一个文件当作“相机 SD 卡”使用: 镜像文件保存了一个文件系统的完整字节序列, loop device 把它伪装成块设备, 文件系统驱动再把它解析成目录树。

2.5 FHS: 根目录的共同语言

Linux 还有一个约定层: Filesystem Hierarchy Standard (FHS)。它规定 /usr、/home、/mnt、/dev 等目录大致应该放什么。FHS 不决定用户文件内容, 却给系统软件、脚本和管理员提供了共同预期。

⁹视频画面时间区间: 00:36:15–00:36:20。

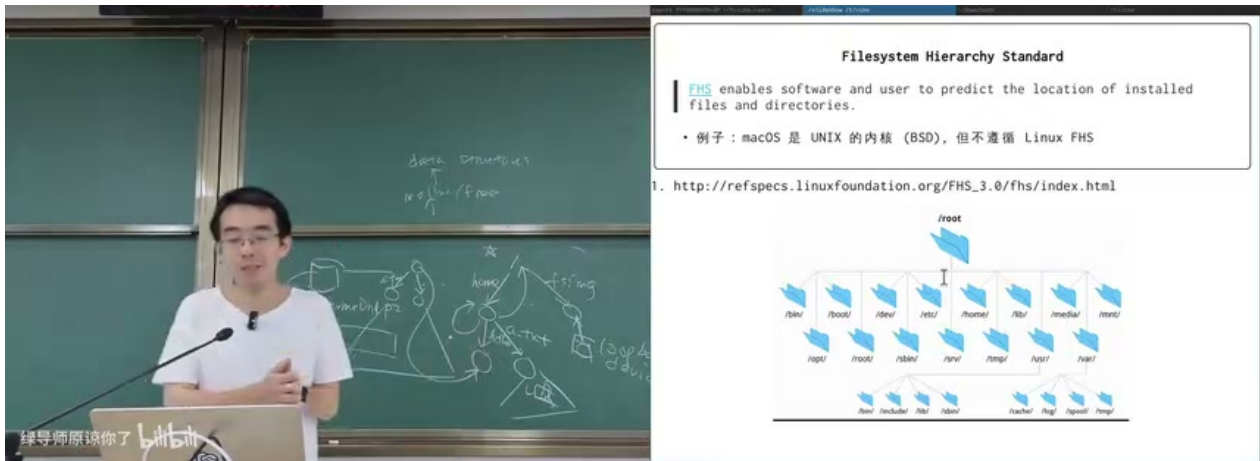


图 10: FHS 规定 Linux 根目录下常见目录的职责, 让软件 and 用户对系统布局形成共同语言。¹⁰

2.6 本章小结

挂载把多棵目录树组合成统一命名空间。Windows 通过盘符和对象命名空间维护兼容性, Unix/Linux 倾向于把一切接到 / 下。块设备、光盘、镜像文件、loop device 和虚拟文件系统的共同点是: 它们都提供某种可被路径遍历的对象树。

3 目录树 API: 增删改查与链接

有了目录树, 自然需要对它增删改查: 创建目录、删除目录、遍历目录、重命名目录项、解析路径。命令行工具的很多“高级能力”都建立在这些朴素系统调用之上。

3.1 遍历与 glob

mkdir 背后是 mkdirat; ls 背后是 getdents64; shell 中的 `**/*.conf` 不是一个内核通配符, 而是 shell 先遍历目录、展开成参数, 再调用目标程序。今天的 AI 工具也类似: Read、Write、Bash、Grep、Glob 都是在文件系统 API 上组合出来的能力。

路径解析是循环

解析 `/a/b/c` 时, 内核从根目录开始查找 `a`, 进入结果目录后查找 `b`, 再查找 `c`。普通目录项返回对象; 挂载点切换到另一棵树; 符号链接可能把剩余路径替换成另一段路径。复杂性都藏在这个循环里。

¹⁰视频画面时间区间: 00:37:30–00:37:35。

3.2 硬链接：多个名字指向同一个文件

如果目录项是名字到对象的指针，那么多个目录项完全可以指向同一个普通文件。这就是硬链接。硬链接不复制文件内容，两个名字共享同一个 inode、同一份数据和同一组元数据。删除其中一个名字时，系统调用叫 `unlink`，它减少链接计数；只有链接计数归零且没有打开引用时，文件内容才可以回收。

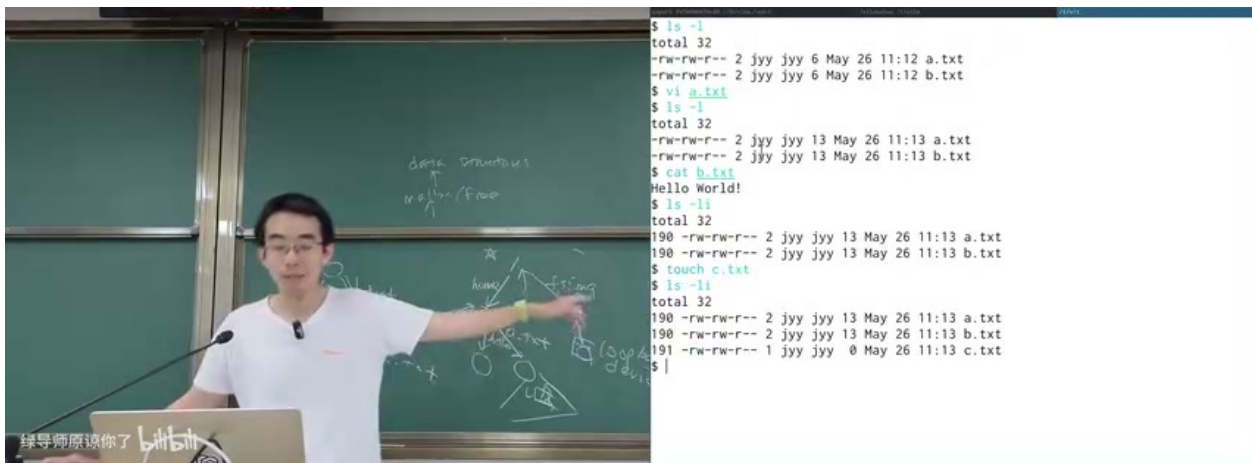


图 11: 硬链接演示：a.txt 和 b.txt 显示相同 inode 号，修改其中一个名字看到的是同一份文件内容。¹¹

硬链接为什么不能随意链接目录

若普通用户可以给目录创建硬链接，目录图就可能形成环。引用计数无法回收与根目录树脱离但内部互相引用的环，因此 Unix 通常禁止普通硬链接指向目录，`.` 和 `..` 是文件系统维护的特殊例外。

3.3 符号链接：把路径写进一个特殊文件

符号链接更像快捷方式：符号链接对象存储一段路径字符串。路径解析遇到符号链接时，把这段字符串纳入解析流程。它可以跨文件系统，可以指向目录，也可以形成环；如果目标路径不存在，就得到 dangling symlink。

¹¹ 视频画面时间区间：00:43:50–00:43:55。

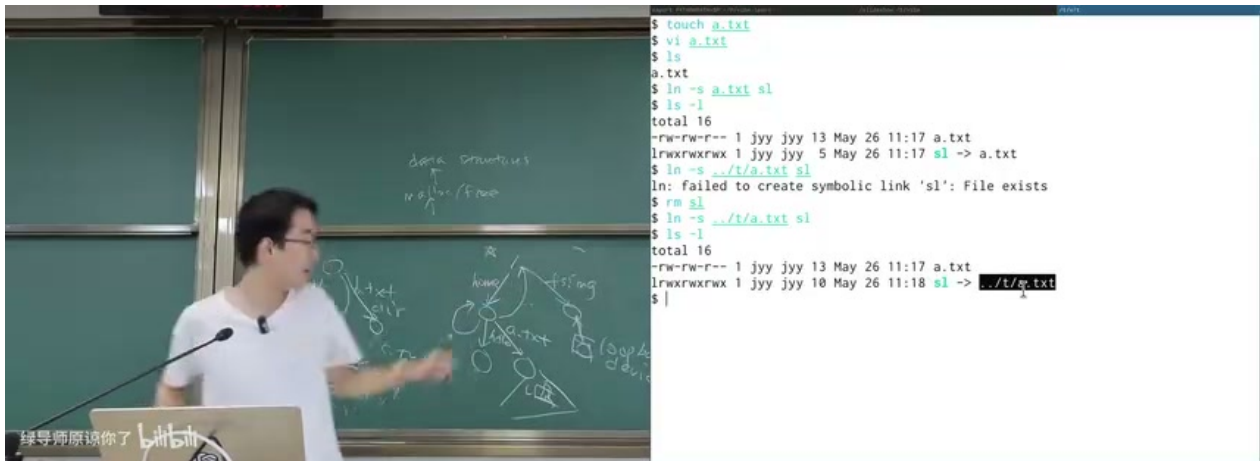


图 12: 符号链接演示: `sl -> ../t/a.txt` 存储的是路径字符串; 删除目标后链接对象仍在, 但解析会失败。¹²

符号链接的威力在于“伪造”目录树: 实体文件可以放在一个真实位置, 用户看到的目录结构可以由一组链接拼出来。课堂中讲者用符号链接构造文字冒险游戏的状态图, 又进一步引出 Nix。

3.4 Nix: 用链接拼出可回滚的软件世界

Nix 把软件包的不同版本放在 `/nix/store` 中, 每个目录名包含内容 hash 和包名。用户环境或系统环境则通过链接把某一组版本拼成当前可见的目录树。由于 store 中对象近似只增不改, 旧环境可以长期保留, 新环境可以快速生成, 回滚也只是切换链接。

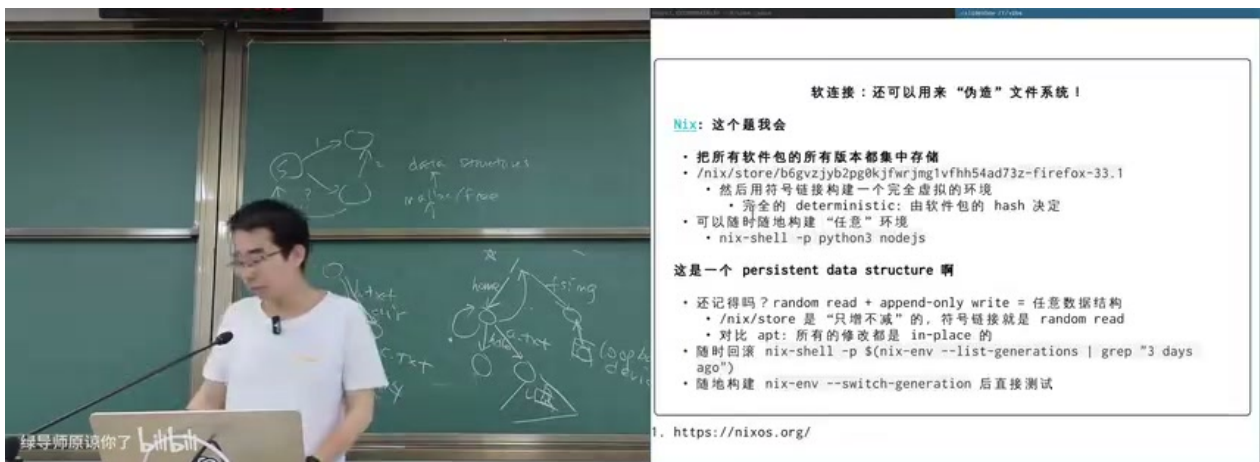


图 13: Nix 的核心直觉: 所有版本存入内容寻址的 store, 用符号链接把某一代环境拼成用户可见的文件系统。¹³

¹²视频画面时间区间: 00:47:20–00:47:25。

¹³视频画面时间区间: 00:52:50–00:52:55。

只增不改与持久化数据结构

如果底层支持随机读和 append-only 写，就可以用“复制路径、共享未变子树”的方式实现可持久化数据结构。Nix store 的工程味道类似：旧对象不被原地覆盖，新 generation 通过链接选择一组对象。

3.5 本章小结

目录树 API 的基本操作很少，但组合能力很强。硬链接让多个名字共享同一对象，符号链接让路径解析跳到另一段路径，Nix 进一步把链接机制变成软件环境管理的基础。理解目录项是“带名字的指针”，许多 Unix 行为就不再神秘。

4 文件元数据、权限与虚拟文件系统

文件不只是字节序列。它还有类型、权限、所有者、组、时间戳、大小、链接数、扩展属性，以及文件系统特有的行为。这些元数据决定了文件如何被显示、打开、执行、复制和同步。

4.1 ls -l: 属性的压缩视图

ls -l 的一行输出压缩了很多属性：文件类型、mode 位、链接数、owner、group、大小、修改时间和名字。第一列第一个字符表示类型：- 是普通文件，d 是目录，l 是符号链接，p 是管道，c 是字符设备，b 是块设备。后九位分成 user、group、other 三组，每组是 read/write/execute。

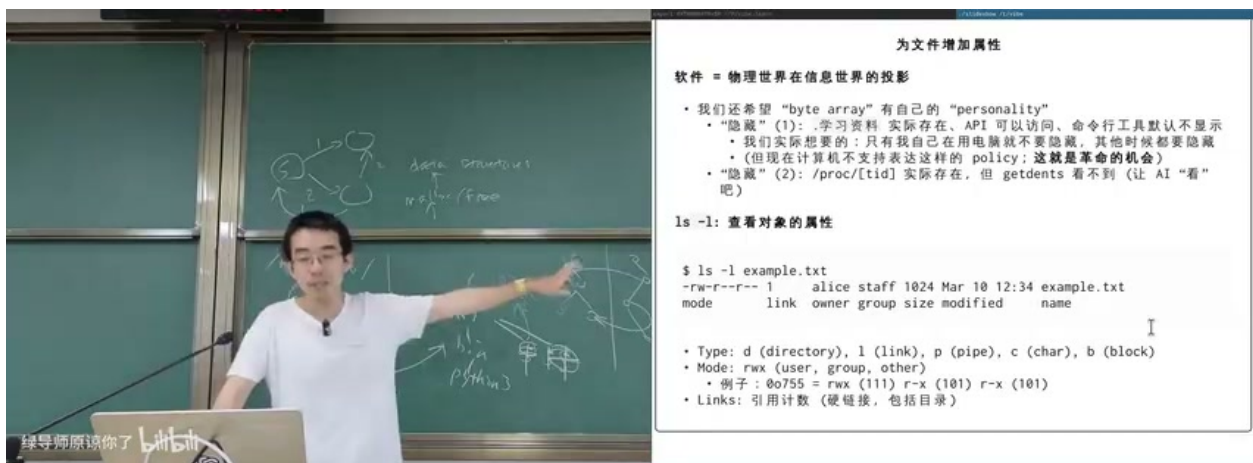


图 14: ls -l 暴露文件属性：类型、权限位、链接数、所有者、组、大小和时间戳共同影响系统行为。¹⁴

¹⁴视频画面时间区间：01:03:15–01:03:20。


```

1 -rw-r--r--  owner can read/write
2 drwxr-xr-x  directory, owner can enter/write, others can enter/read
3 lrwxrwxrwx  symbolic link; real access checks apply to resolved target
4 chmod 755 file
5 chmod 000 file

```

Listing 2: mode 位的常见解释

root 与权限检查

Unix 的 0 号用户具有绕过多数普通权限检查的能力。若 root 都无法删除或修复某个对象，系统可能被普通用户构造的对象占满或锁死。因此权限机制必须和“系统可管理性”一起设计。

4.2 文件系统可以“说谎”：procfs

普通磁盘文件系统把目录项和文件内容保存在持久设备上，但 procfs 是代码生成的目录树。你看到 /proc/<pid>/status、/proc/<pid>/mem、/proc/mounts，并不意味着磁盘上真的有这些文件。内核在 open、getdents64、read 等接口被调用时，临时从内核数据结构生成结果。

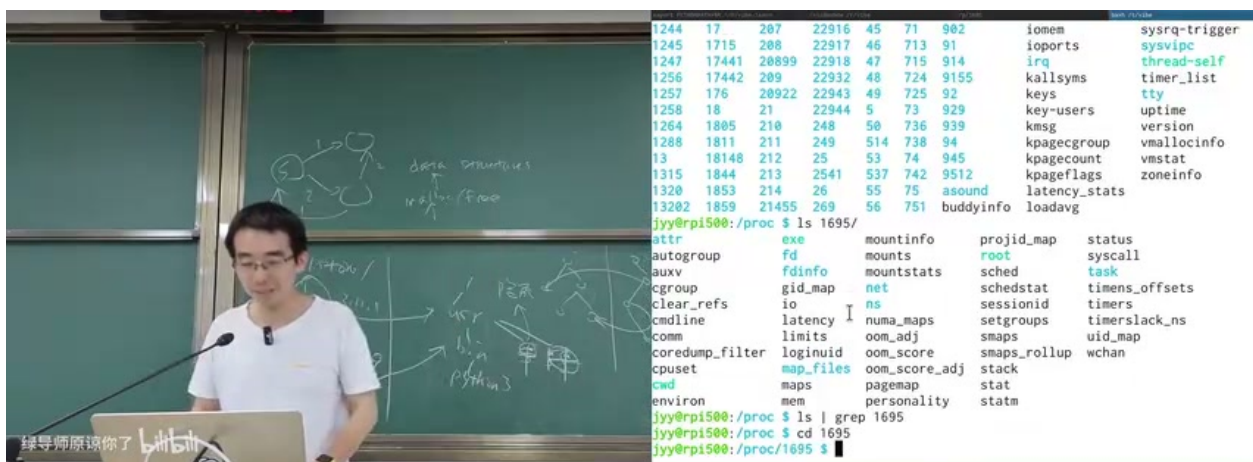


图 15: /proc 中的进程目录可以被直接访问，却可能不会出现在一次普通 `ls | grep` 中；虚拟文件系统能按自己的策略生成目录项。¹⁵

这说明文件系统 API 是一种协议：只要 open、read、getdents64 等返回值符合预期，上层程序就会把它当成文件系统。底层对象可以来自磁盘、内核结构、网络、数据库或现场计算。

¹⁵ 视频画面时间区间：01:07:00–01:07:05。

4.3 Extended Attributes: 给文件附加 key-value

传统 Unix 元数据固定而有限。Extended Attributes (xattr) 给每个文件附加命名空间化的 key-value, 例如 `user.author`、`user.description`、`com.apple.metadata:kMDItemWhereFroms`。它可以存下载来源、签名、标签、摘要、应用私有状态, 甚至为未来的语义搜索提供索引。

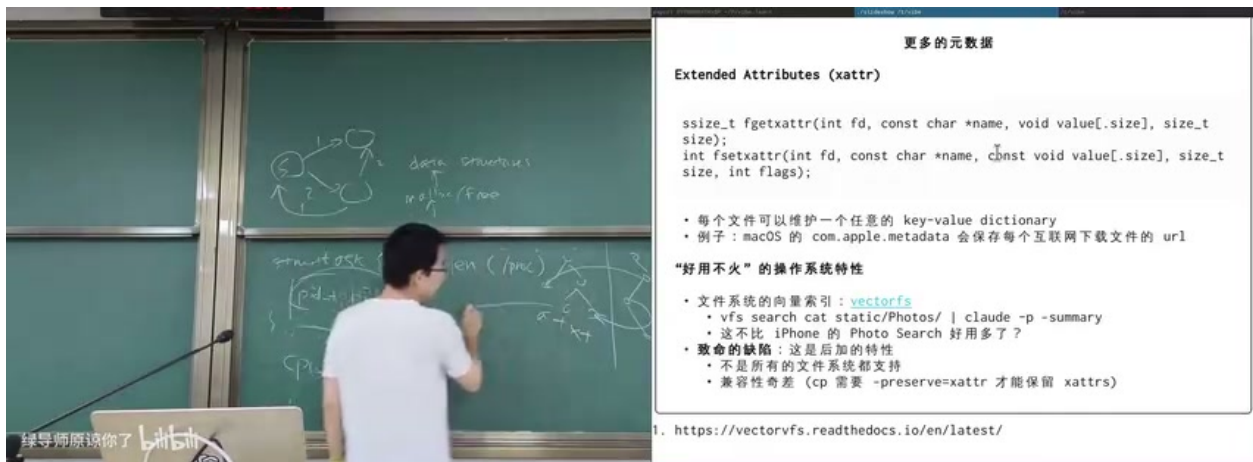


图 16: xattr 把文件元数据扩展为 key-value 字典; 同时也带来复制、同步和跨文件系统兼容问题。¹⁶

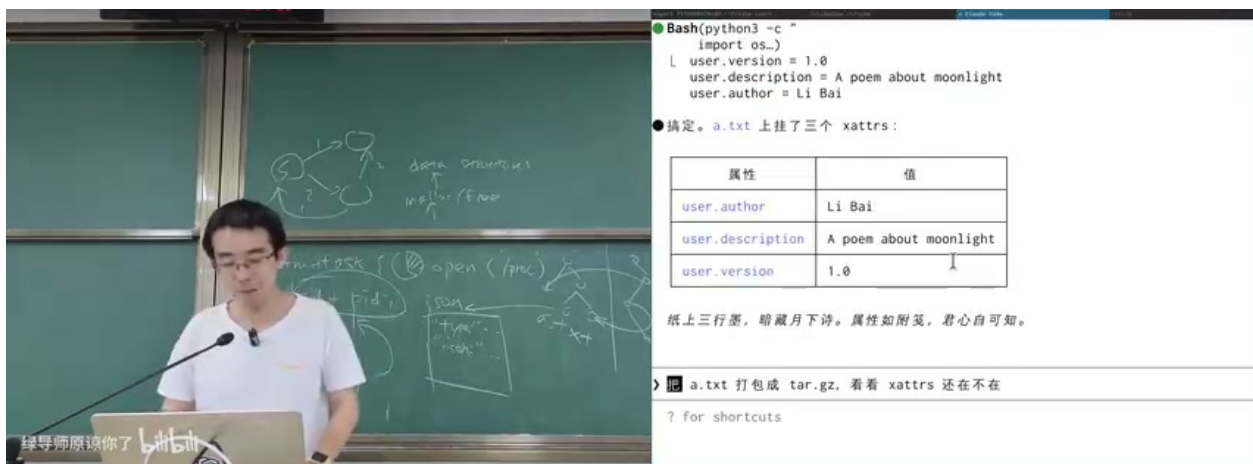


图 17: 课堂演示为 `a.txt` 添加 `user.author`、`user.description`、`user.version` 等扩展属性。¹⁷

xattr 最大的坑是“静默丢失”

不是所有文件系统和拷贝工具都保留 xattr。把带 xattr 的文件复制到 FAT 文件系统、普通 tar 包或不支持 xattr 的同步工具中, 扩展属性可能悄悄消失。文件内容相同, 不代表文件语义完全相同。

¹⁶ 视频画面时间区间: 01:10:35–01:10:40。

¹⁷ 视频画面时间区间: 01:12:10–01:12:15。

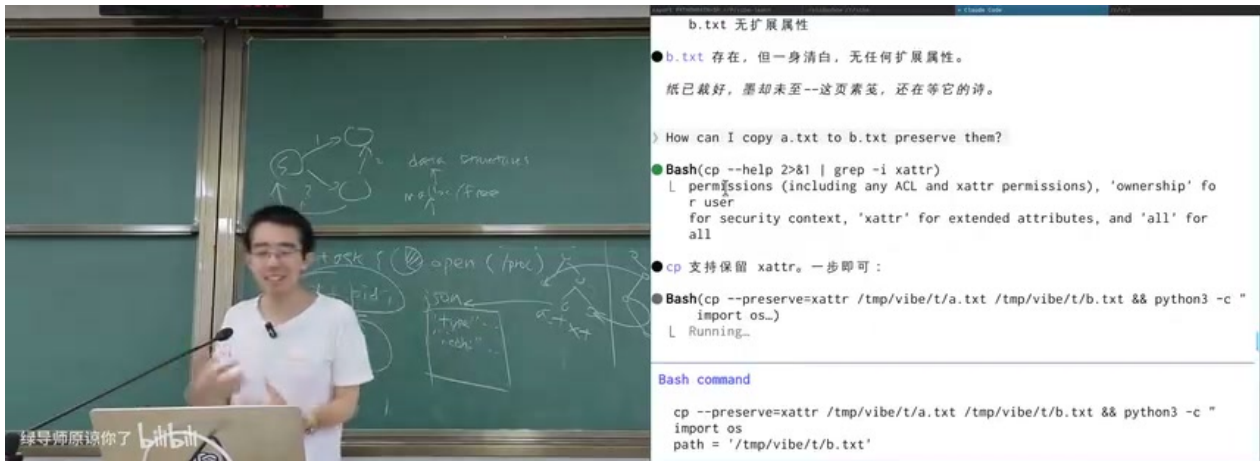


图 18: 保留 xattr 需要工具显式支持，例如 `cp --preserve=xattr`；否则复制后的文件可能只有内容，没有扩展属性。¹⁸

4.4 ACL: 比 user/group/other 更细的授权

三组权限位简单、稳定、易实现，但表达能力有限。Access Control List (ACL) 允许为特定用户或组设置更细粒度的访问规则，例如允许某个用户读、拒绝某个用户写、让多个组拥有不同权限。

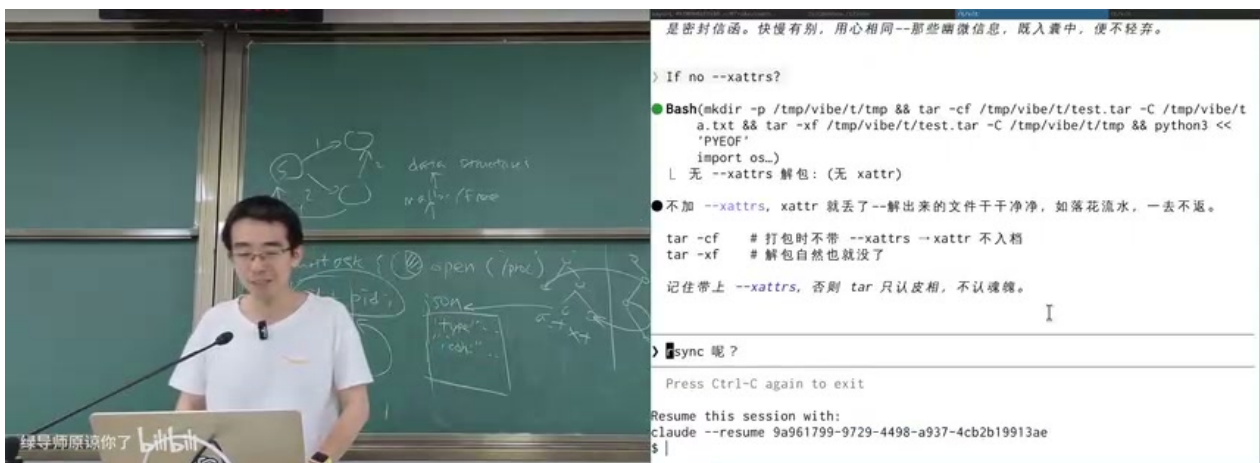


图 19: ACL 扩展了 user/group/other 模型，可为特定用户或组设置更细粒度的访问控制。¹⁹

ACL 的意义不只是“多几个权限位”。它体现了文件系统 API 的持续演化：早期 FAT 没有 Unix 权限；Unix 引入 owner/group/mode；现代系统又叠加 xattr、ACL、签名、隔离标记、云同步状态。文件越来越像一个带有行为和策略的对象，而不仅是字节。

¹⁸视频画面时间区间：01:12:45–01:12:50。

¹⁹视频画面时间区间：01:17:00–01:17:05。

4.5 本章小结

文件元数据决定文件如何被访问、显示、执行、复制和同步。`procfs` 说明文件系统可以由代码动态生成，`xattr` 说明元数据可以扩展成 key-value，ACL 说明访问控制可以比三组 mode 位更细。文件系统 API 的稳定外表下，实际语义一直在增长。

5 总结与延伸

这节课的主线可以压缩为一句话：**文件系统是持久化世界里的抽象数据类型，它把块设备、内核对象和外部资源统一投影为可命名、可遍历、可授权的对象树。**

从实现角度看，本讲给出了五个关键机制：

- **文件**：可变长字节序列，是普通用户最熟悉的虚拟磁盘；
- **目录**：名字到对象的映射表，是路径解析的基本节点；
- **挂载**：把多棵目录树粘接到统一命名空间；
- **链接**：让名字和对象解耦，硬链接共享对象，符号链接重写路径；
- **元数据**：让对象拥有权限、类型、属性和策略。

从系统设计角度看，文件系统有三层角色。第一层是**存储分配器**：在块设备上管理空间、目录和文件内容。第二层是**命名服务**：用路径把人类可理解的名字映射到对象。第三层是**对象协议**：不论对象来自磁盘、内核、网络还是程序生成，只要它响应文件系统 API，上层软件就能用同一套工具组合它。

本讲最重要的抽象迁移

不要把文件系统理解为“磁盘上的文件夹”。更好的理解是：文件系统是一套把对象组织成目录树的协议。磁盘文件只是其中一种对象；块设备、进程信息、光盘、镜像文件、符号链接、Nix 环境、`xattr` 和 ACL 都是这套协议的不同侧面。

讲者最后强调，今天的工具环境让探索这些机制变得容易：你不必记住每个系统调用的全部细节，但必须知道世界上存在 `mount`、`ioctl`、`loop device`、`xattr`、ACL 这样的机制，并能在需要时让工具帮助你读手册、跑实验、验证行为。操作系统课真正训练的是抽象边界感：知道 API 承诺了什么、没有承诺什么，以及怎样把这些机制可靠地组合进自己的系统。

5.1 本章小结

本讲从块设备出发，逐步构造了文件系统 API 的人类视角：文件是虚拟磁盘，目录树提供局部性和命名，挂载合并多棵树，链接让名字和对象解耦，元数据和 ACL 则把权限与策略附加到对象上。下一讲转向程序和 AI agent 更常使用的文件系统 API：在那里，路径名仍然重要，但自动化工具会更关心批量遍历、语义搜索、增量更新和可组合的对象协议。

5.2 拓展阅读

- `man 2 mount`: 理解 `mount` 系统调用的参数和错误;
- `man 7 xattr`: 理解扩展属性的命名空间、大小限制和继承问题;
- `man 5 proc`: 理解 `procfs` 如何把内核数据结构投影成文件系统;
- Filesystem Hierarchy Standard: 理解 Linux 根目录布局背后的约定。